

AI芯片与系统知识点整理

考试复习导航

范围：考试覆盖 1-16 所有 PPT。第 15 章中的 FlashAttention 部分不考，明显偏前沿研究介绍、扩展阅读、产业案例细节的内容优先级较低。复习时建议以第 16 章总复习 PPT 为主线，再回到对应章节补细节；时间有限时可以优先看总复习，但不能只背总复习。

高优先级：第 16 章最后一节课出现的四类例题一定要能独立做：Roofline、pipelined CPU / Tomasulo 时序、performance analysis、cache。考试不会照搬原题，所以要掌握建模步骤和判断依据。

额外重点：

主题	必须掌握到什么程度	对应章节
Tomasulo	会解释 register renaming table 与 reservation station 的字段含义； 会按周期判断何时分配、何时写 tag/value、何时 CDB 广播、何时清空或更新表项	第 3 章，第 16 章流水线例题
Tensor parallel	会区分 row-wise / column-wise 切权重，知道 forward/backward 中需要 AllGather、ReduceScatter 或 AllReduce 的位置	第 14 章，第 16 章并行训练回顾
Alternating partitioning	会说明为什么交替使用切分方式能减少层间同步，以及什么时候仍需要 AllReduce	第 14 章
Ring AllReduce	会写出两个阶段、轮数和通信量：ReduceScatter $N-1$ 轮，AllGather $N-1$ 轮，每轮每卡传 M/N ，每卡总发送/接收约 $2(N-1)M/N$	第 14 章
Cache 题	会算 block number、set index、tag，按 direct mapped / set-associative / fully associative 分析 hit/miss 与 compulsory/conflict/capacity miss	第 8-10 章，第 16 章 cache 例题

不建议把精力放在：FlashAttention 具体算法流程；第 15 章前沿优化细节；只用于介绍趋势的超大系统型号和新闻式案例。它们可以帮助理解课程背景，但不应替代基础题训练。

第一章：课程导论、体系结构基础与流水线 CPU

1.1 AI 芯片与系统的课程主线

定义：AI 芯片与系统研究从算法、框架、运行时、编译、计算库、存储、互联到硬件微架构的完整协同设计，目标是在给定能耗、成本和开发复杂度下获得更高性能。

拓展：深度学习爆发依赖算法、数据和算力三要素；当摩尔定律放缓而模型计算需求持续增长时，系统和芯片设计成为 AI 能否落地的关键。课程先补计算机体系结构基础，再进入 GPU、存储层次、缓存一致性、AI 加速器、CANN 和 MindSpore 等内容。

考点：AI 芯片不是孤立硬件，而是应用、框架、运行时、编译、存储和微架构的协同结果；学习目标是理解基础、设计原则和已有先例。

1.2 Amdahl 定律与 Roofline 模型

定义：Amdahl 定律描述固定工作负载下并行化带来的理论加速比，若可并行比例为 f 、处理器数为 N ，则加速比约为 $\frac{1}{(1-f)+f/N}$ 。

拓展：串行部分 $(1 - f)$ 限制最大加速比，即使并行资源无限，最大加速比也不会超过 $\frac{1}{1-f}$ 。Roofline 模型用算术强度 $AI = \frac{Total\ Flops}{Total\ Memory\ Bytes}$ 判断程序受计算峰值还是内存带宽限制，性能上界为 $\min(Peak\ Flop/s, AI \times Peak\ GB/s)$ 。在 log-log 坐标中，Roofline 的斜线部分由 $Performance = AI \times Bandwidth$ 给出，其斜率表示内存带宽；水平部分表示处理器峰值算力。斜线与水平线的交点称为 ridge point，对应 $AI^* = \frac{Peak\ Flop/s}{Peak\ Bandwidth}$ ，用于区分 memory-bound 和 compute-bound 区域。

考点：Amdahl 定律强调串行瓶颈；Roofline 模型强调算术强度、峰值算力和峰值带宽；斜线段斜率越大表示可用内存带宽越高，同一算术强度下可达到的带宽受限性能越高；AI 小通常 memory-bound，AI 大通常 compute-bound。

1.3 Little 定律与硬件并发

定义：Little 定律可写为 $Concurrency = Throughput \times Latency$ ，在硬件中常用来估算为了隐藏长延迟需要多少并发请求或缓冲空间。

拓展：内存子系统中若吞吐为 12GB/s、访问延迟为 100ns，则维持满吞吐需要约 $12GB/s \times 100ns = 1200B$ 的在途数据。延迟越长、目标吞吐越高，系统需要越多 outstanding requests、队列或 buffer。

考点：Little 定律解释为什么现代 CPU/GPU 需要多线程、多请求队列和大量缓冲；它是理解内存级并行和延迟隐藏的重要工具。

1.4 冯·诺依曼模型与 ISA

定义：冯·诺依曼模型由存储器、处理单元、控制单元、输入和输出组成，核心特征是存储程序和顺序指令执行。

拓展：ISA 是软件命令和硬件执行之间的接口，规定指令、寄存器、数据类型、寻址方式和可见状态。MIPS/RISC 类 ISA 常把指令分为计算指令、访存指令和控制流指令；不同寻址方式体现程序员便利性与硬件复杂度之间的权衡。

考点：ISA 是架构，不是具体实现；程序员可见状态包括 PC、寄存器和内存；复杂指令能提高代码密度但增加硬件实现复杂度。

1.5 ISA 的三个组成部分

定义：课件中将 ISA 的内容概括为三部分：存储组织、寄存器集合和指令集。

拓展：存储组织规定地址空间、寻址粒度和是字寻址还是字节寻址；MIPS 地址空间为 2^{32} ，地址粒度为 8 bit，实际是 byte-addressable。寄存器集合规定程序员可见的寄存器数量和用途，MIPS 有 32 个通用寄存器。指令集规定 opcode、数据类型和寻址方式，用来覆盖程序执行所需的基本任务。

考点：MIPS 是字节寻址，不是简单的字寻址；ISA 规定软件能看到什么，微架构决定硬件如何实现这些规定。

1.6 ABI 与 MIPS 寄存器约定

定义：ABI (Application Binary Interface) 是二进制程序模块之间的接口约定，规定函数调用、寄存器使用、参数传递和返回值等规则。

拓展：MIPS ABI 中 `$0` 恒为 0；`$v0-$v1` 用于函数返回值；`$a0-$a3` 用于函数参数；`$t0-$t9` 是临时寄存器；`$s0-$s7` 是需要保存的寄存器；`$gp` 是全局指针，`$sp` 是栈指针，`$fp` 是帧指针，`$ra` 保存函数返回地址。高级语言程序员通常不直接关心 ABI，但汇编程序员必须遵守 ABI，否则模块之间无法正确协作。

考点：ABI 不是硬件电路本身，而是软件二进制层面的约定；函数调用中 `$ra`、`$sp`、参数寄存器和返回值寄存器最常考。

1.7 寻址方式与数据类型

定义：寻址方式说明操作数位于哪里以及有效地址如何计算；数据类型说明 ISA 支持什么形式的数据。

拓展：MIPS 常见寻址方式包括立即数寻址、寄存器寻址、PC-relative、pseudo-direct 和 base+offset。立即数寻址中操作数直接在指令字段里，寄存器寻址中操作数在寄存器中，base+offset 常用于 load/store，PC-relative 常用于条件分支，pseudo-direct 常用于 jump。MIPS 支持二进制补码整数、无符号整数和浮点数；早期 RISC 机器可能只支持整数，而 AI 芯片可能把 tensor 作为 ISA 中更高语义层次的数据类型。

考点：更多寻址方式可减少指令数和代码大小，便于表示指针、数组和稀疏访问，但会增加编译器和微架构复杂度；数据类型越高层，语义鸿沟越小，但硬件实现越复杂。

1.8 指令格式、Opcode 与 Operand

定义：一条指令由 opcode 和 operands 构成。Opcode 说明“做什么”，operands 说明“对谁做”。

拓展：二者都由指令格式或指令编码指定。MIPS 指令为 32 bit，例如 R-type 指令包含 `op`、`rs`、`rt`、`rd`、`shamt`、`funct` 等字段，其中 `op` 指定主操作码，寄存器字段指定源操作数和目的操作数。`add $s0, $s1, $s2` 可理解为 `rd <- rs + rt`；`addi $s0, $s1, 5` 使用立即数字段并执行 `rt <- rs + sign-extend(imm)`。

考点：opcode 决定操作类型，operand 字段决定源/目的位置；MIPS 中立即数通常需要符号扩展；目的操作数字段位置也会影响译码和硬件效率。

1.9 MIPS 三类主要指令

定义：MIPS 指令可分为计算指令、数据搬运指令和控制流指令。

拓展：计算指令在 ALU 中完成算术或逻辑运算，如 `add`、`and`、`nor`、`xor` 及其立即数版本，也包括浮点 F-type 操作。数据搬运指令在寄存器和内存之间搬运数据，`lw` 和 `sw` 使用 base+offset 寻址，例如 byte-addressable MIPS 中访问 `A[2]` 需要偏移 8 字节。控制流指令改变 PC，包括条件分支和无条件跳转；`beq` 满足条件时执行 `PC <- PC + sign-extend(offset) * 4`，`j` 使用 pseudo-direct 寻址，`jr` 使用寄存器寻址，`jal` 用于函数调用并保存返回地址。

考点：operate 指令不直接访问内存，load/store 才访问内存；byte-addressable 下数组 word 下标要乘以 4；分支和跳转的本质是改变 PC。

1.10 ISA 设计权衡

定义：ISA 设计需要在表达能力、代码密度、编译器复杂度、硬件复杂度、性能和能耗之间权衡。

拓展：复杂指令一次完成更多工作，例如链表插入、FFT 或矩阵乘法，优点是编码更紧凑、代码更短、指令 cache 利用率可能更高，编译器负担较小；缺点是硬件译码和控制更复杂，编译器进行细粒度优化的机会更少。简单指令只完成原语操作，硬件规整、流水线友好，但可能需要更多指令表达复杂操作。寄存器数量也是 ISA 权衡：寄存器多有利于编译器分配、减少保存/恢复和访存，但会增加寄存器编号位数、指令长度、寄存器文件面积、访问时间和功耗。

考点：x86、VAX 等偏复杂指令，MIPS、ARM、RISC-V 等偏 RISC 风格；AI 芯片的 tensor 指令体现“领域专用高语义指令”；寄存器越多不一定越好，要看编码成本和硬件代价。

1.11 单周期、多周期与流水线处理

定义：指令处理通常包含取指 IF、译码/读寄存器 ID、执行 EX、访存 MEM、写回 WB。单周期 CPU 每条指令一个周期完成，多周期 CPU 将一条指令拆成多个周期，流水线 CPU 将不同指令的不同阶段重叠执行。

拓展：单周期 CPU 的 CPI 为 1，但时钟周期受最慢指令关键路径限制；多周期 CPU 缩短单周期时间，但同一时刻硬件并发有限；流水线提高吞吐，但引入结构冒险、数据冒险和控制冒险。

考点：单周期优化延迟简单但频率低；多周期按需使用周期；流水线提高吞吐而不一定降低单条指令延迟。

第二章：流水线冒险与重排序缓冲

2.1 流水线冒险分类

定义：流水线冒险是阻止指令按预定阶段继续执行的条件，主要包括结构冒险、数据冒险和控制冒险。

拓展：结构冒险来自同周期多个指令竞争同一硬件资源，例如寄存器文件端口不足或指令/数据存储端口冲突。数据冒险来自指令间依赖，包括 RAW、WAR、WAW，其中 RAW 是真实数据依赖；控制冒险来自分支和跳转导致 PC 依赖。

考点：结构冒险靠资源复制或分离端口解决；RAW 不能通过简单重命名消除；WAR/WAW 是假依赖，可通过重命名消除。

2.2 停顿、NOP 与数据转发

定义：流水线停顿通过冻结部分流水级并插入 bubble 来等待依赖满足；数据转发是在结果写回寄存器前直接把结果送给后续依赖指令。

拓展：硬件停顿需要 PC、流水线寄存器 enable/clear 和依赖检测逻辑。编译器可插入 NOP 或调度独立指令，但静态调度难处理上下文相关延迟。转发通常从 MEM 或 WB 阶段转发到 EX 阶段，但 load-use 这类场景可能仍需停顿。

考点：转发减少 RAW 导致的停顿，但不是万能；软件调度是静态调度，硬件调度可根据真实运行情况动态处理。

2.3 精确异常与中断

定义：精确异常要求处理异常或中断时，体系结构状态看起来像按程序顺序执行到某条指令边界：之前指令全部完成，之后指令没有改变可见状态。

拓展：单周期和多周期机器较容易保持精确异常；流水线和乱序执行可能出现后续指令先完成的问题，需要延迟提交或按序退休机制恢复顺序语义。精确异常便于调试、系统调用、虚拟内存缺页处理和程序恢复。

考点：异常通常来自当前线程内部，检测后处理；中断通常来自外部事件，可在指令边界处理；精确异常是现代处理器必须维护的顺序语义。

2.4 重排序缓冲 ROB

定义：重排序缓冲（ROB）允许指令乱序完成，但在提交阶段按程序顺序写回体系结构状态。

拓展：指令译码时分配 ROB 项，执行结果先写入 ROB，只有到达队头并确认无异常时才提交到寄存器或内存。ROB 可支持多周期执行、精确异常和寄存器重命名，从而消除 WAR/WAW 假依赖。

考点：ROB 的核心思想是乱序完成、按序提交；寄存器值可能位于寄存器文件或 ROB；ROB 简化精确异常，但增加访问复杂度和硬件面积。

第三章：Tomasulo 算法与乱序执行

3.1 乱序执行动机

定义：乱序执行让独立指令绕过被真实依赖或长延迟阻塞的指令，动态提高指令级并行度。

拓展：仅有 ROB 的按序发射设计仍可能因为一条等待操作数的指令阻塞整个流水线。保留站把等待的指令移出前端路径，让独立指令可以先进入功能单元执行，从而容忍乘除法、访存等多周期延迟。

考点：乱序执行主要收益来自延迟容忍；若所有操作都是单周期且无缓存未命中，乱序收益会明显下降。

3.2 Tomasulo 算法组件

定义：Tomasulo 算法通过寄存器重命名、保留站和公共数据广播实现动态调度和乱序执行。

拓展：寄存器别名表 (RAT) 记录体系结构寄存器当前对应的生产者 tag；保留站保存等待执行的指令、源操作数值或 tag；功能单元完成后广播结果和 tag，等待该 tag 的消费者被唤醒。该机制动态构造一个有限窗口内的数据流图。若按“两张表”来理解，通常就是维护保留站表和寄存器结果状态表：前者记录每条在窗口中等待的指令，后者记录每个目的寄存器当前将由哪个保留站写回。

拓展：RS (Reservation Station) 阶段要做三件事：第一，给新指令分配空闲保留站项，并写入操作类型、目的寄存器和源操作数字段；第二，检查每个源寄存器当前是否已有未完成生产者，若源操作数已经就绪，就把值直接写入 `vj/vk`，若尚未就绪，就把对应生产者 tag 写入 `qj/qk`；第三，若该指令会写目的寄存器，就在寄存器结果状态表中把该目的寄存器映射到当前保留站 tag，完成寄存器重命名。只有当保留站项里的源操作数都就绪，即 `qj/qk` 为空时，指令才可以被发射到功能单元执行。

拓展：两张表的维护规则可按“分配-等待-广播-释放”记忆。发射到 RS 时，保留站表新建一项并置 Busy，填入 `op`、`vj/vk` 或 `qj/qk`；同时更新寄存器结果状态表，让目的寄存器指向该 RS。等待阶段，保留站持续监听公共数据总线 CDB。某功能单元广播 `<tag, value>` 后，所有保留站项都比较自己的 `qj/qk` 是否等于该 tag；若相等，就把对应 `vj/vk` 写成 value，并清空 `qj/qk`，表示该源操作数已就绪。写回时，若寄存器结果状态表中该目的寄存器仍指向这个 tag，说明它仍是最新生产者，就把表项清空或更新为新 tag；若已被更年轻指令覆盖，则不能回写旧映射。指令完成并离开该保留站后，还要把该保留站表项释放为空闲，供后续指令复用。

考点：tag 将消费者与生产者连接起来；保留站实现等待与唤醒；寄存器结果状态表负责记录“哪个寄存器正在等谁写回”；RS 阶段的核心是分配表项、捕获就绪值、记录未就绪 tag、更新目的寄存器映射；WAR/WAW 通过重命名消除，RAW 通过等待 tag 广播解决。

考试强化：Tomasulo 表格题通常考“两张表何时变化”。可以把 register renaming table 看成每个架构寄存器的一条状态记录，把 reservation station 看成每条已进入调度窗口、但还未执行完成/释放的指令记录。

数据结构	常见字段	字段含义	什么时候更新
Register renaming table / RAT / register result status	<code>valid</code> 、 <code>Tag</code> 、 <code>value</code> ，或只写 <code>Qi</code>	<code>valid=1</code> 表示寄存器文件当前值可直接用； <code>valid=0</code> 表示该寄存器的新值还没产生，要等 <code>Tag/Qi</code> 指向的 RS 或 ROB； <code>value</code> 是已提交或当前可用值	发射/rename 写目的寄存器时，把目的寄存器改成 <code>valid=0, Tag=当前 RS/ROB tag</code> ；CDB 广播时，若表中 tag 与广播 tag 相同，写入 value 并置 <code>valid=1</code> 或清空 tag；若已被更年轻指令覆盖，不更新

数据结构	常见字段	字段含义	什么时候更新
Reservation station / RS	Busy、Op、Dest/Tag、Vj/Vk、Qj/Qk， 或 source.valid/source.value/source.tag	保存一条等待执行的指令。 Vj/Vk 是已就绪源操作数值； Qj/Qk 是未就绪源操作数等待的生产者 tag； Dest/Tag 是本指令结果广播时使用的名字	issue 时分配空项并填 Op；读源寄存器，若 RAT valid 则写 Vj/Vk，否则写 Qj/Qk；监听 CDB，若 Qj/Qk 命中广播 tag，则写 value 并清空对应 Q；两个源都 ready 后发射到 FU；结果广播/写回后释放 RS
Common Data Bus / CDB	<tag, value>	功能单元完成后对所有等待者广播“哪个生产者的值好了”	执行完成时广播；所有 RS 比较自己的 Qj/Qk，RAT/RF 比较自己的 Tag/Qi，命中者同时更新

推表顺序：每个周期先确认哪些指令完成并广播，再更新所有等待该 tag 的 RS 和 RAT；再判断哪些 RS 操作数齐全、可进入执行；最后按题目设定的 issue/dispatch 宽度给新指令分配 RS 和重命名目的寄存器。若题目给定另一种周期顺序，以题目为准，但必须在答案里保持同一顺序。

常见陷阱：WAW 不是让老指令覆盖新指令的理由。老指令广播时，只有 RAT 中仍指向老 tag，才说明它还是该架构寄存器的最新生产者；如果 RAT 已经指向更年轻指令的 tag，老结果只能唤醒等待它的消费者，不能改写 RAT 的最新映射。RAW 不能被重命名消除，只能通过等待 Qj/Qk 对应 tag 的广播解决。WAR/WAW 是名字相关，可以通过目的寄存器重命名消除。

3.3 两个窗口：调度窗口与提交窗口

定义：现代乱序处理器常有两个关键“窗口”：保留站/调度窗口负责乱序发射执行，ROB/提交窗口负责按序退休。

拓展：前者让指令在操作数就绪后尽快执行，后者保证体系结构状态按程序顺序更新。两者结合形成“乱序执行 + 精确异常”的现代处理器基本结构。

考点：保留站解决执行顺序问题，ROB 解决提交顺序问题；两者分别对应性能和正确性。

3.4 乱序执行的代价

定义：乱序执行的硬件代价包括寄存器重命名表、保留站、唤醒选择逻辑、广播网络、ROB 和复杂控制逻辑。

拓展：tag 广播、值捕获和指令唤醒可能成为关键路径；窗口越大，比较器、端口和互联复杂度越高。实际处理器会在窗口大小、频率、功耗和性能之间折中。

考点：乱序执行不是免费提升 IPC；其收益依赖程序 ILP 和长延迟事件，代价体现在面积、功耗、时钟频率和验证难度上。

第四章：超标量、SIMD、多线程与多核

4.1 超标量执行

定义：超标量处理器每个周期取指、译码、执行和提交多条指令，目标是提高 IPC。

拓展：N-wide 超标量需要更多功能单元、更宽前端、更复杂依赖检查和更高带宽寄存器文件。按序超标量容易被依赖限制，乱序超标量则通过调度窗口挖掘更多 ILP。

考点：超标量提高指令吞吐，但依赖检查复杂度随宽度上升；其对 Roofline 的影响主要体现在提高峰值算力。

4.2 Flynn 分类与 SIMD

定义：Flynn 分类包括 SISD、SIMD、MISD 和 MIMD。SIMD 是一条指令作用于多个数据元素，适合数据并行任务。

拓展：AVX2 可一次处理 8 个 FP32，AVX-512 可一次处理 16 个 FP32。支持 SIMD 需要向量寄存器文件、向量 ALU、向量访存路径和对应 ISA。SIMD 常用于科学计算、图形视频处理、矩阵运算和大数据处理。

考点：SIMD 提升并行部分吞吐，但受 Amdahl 串行部分和内存带宽限制；SIMD 是显式向量指令模型。

4.3 细粒度多线程

定义：细粒度多线程在硬件中保存多个线程上下文，并按周期切换线程发射指令，以隐藏流水线、分支或访存延迟。

拓展：每个线程的 PC 和寄存器独立，硬件可在某线程等待时调度其他线程。优点是降低单线程相关性检查和分支预测压力，缺点是单线程延迟可能变高，且需要足够线程数量维持吞吐。

考点：多线程通过时间复用硬件隐藏延迟；它改善吞吐和资源利用率，不等同于提高单线程 IPC。

4.4 多核与大小核权衡

定义：多核处理器在同一芯片上集成多个核心，通过线程级并行提高整体吞吐。

拓展：相比继续做更宽、更复杂的单核，多核用较简单核心复制获得更好的能效和可扩展性。大核适合串行代码和低延迟任务，小核适合高并行吞吐任务；非对称多处理器结合大核和小核，试图兼顾串行性能与并行吞吐。

考点：多核选择来自功耗墙、复杂度墙和超标量收益递减；并程序可用多核加速，串行部分仍受 Amdahl 定律限制。

第五章：存储系统、SRAM、DRAM 与 HBM

5.1 理想存储与现实矛盾

定义：理想存储应具有零延迟、无限容量、无限带宽和零成本，但现实中容量、速度、带宽、能耗和成本相互矛盾。

拓展：通常越大越慢、越快越贵。寄存器和 SRAM 速度快但容量小，DRAM 容量大但延迟高，SSD/硬盘容量更大但延迟更长。AI 等数据密集型负载常被存储容量、带宽和数据搬运能耗限制。

拓展：从容量看，典型层次可粗略记为 SRAM 约 10MB，HBM 约 10GB，DDR/DRAM 约 100GB，SSD 约 1TB，磁盘约 10TB，越靠下容量越大。从带宽看，SRAM 可达约 1TB/s，HBM 约 100GB/s，DDR 约 10GB/s，SSD 约 1GB/s，磁盘约 10MB/s，越靠近计算单元带宽越高。从延迟看，SRAM 约 1ns，HBM/DDR 约 100ns，SSD 约 1us，磁盘约 1ms，越远离计算单元延迟越高。

考点：现代系统大部分能耗和时间常花在移动数据而非计算；存储优化常优先追求容量和带宽，而不是单纯降低绝对延迟；容量、带宽、延迟三者不能同时最优，存储层次就是在小而快、大而慢之间折中。

5.2 FF、SRAM、DRAM 与 Flash 对比

定义：Flip-Flop、SRAM、DRAM 和 Flash/SSD 位于不同存储层次，主要差别体现在速度、容量、成本、是否易失以及访问粒度。

存储类型	制作工艺/存储单元	单 bit 成本	典型容量量级	典型带宽量级	典型访问延迟	主要特点与用途
Flip-Flop (FF)	由多个逻辑门/晶体管构成的时序单元，直接集成在逻辑电路中	最高，1 bit 需要较多晶体管	KB 级	最高，可高度并行访问	亚 ns 到 ns 级	用于寄存器、流水线寄存器和控制状态，速度最快但面积最贵
SRAM	典型 6T 单元，由交叉耦合反相器保存 1 bit，CMOS 逻辑工艺兼容	很高，1 bit 通常 6 个或更多晶体管	MB 级，片上可到数十 MB	约 TB/s 级	约 1ns	用于 cache、scratchpad、片上 buffer，速度快、无需刷新，但容量有限
DRAM/DDR	1T1C 单元，用一个晶体管加一个电容存储电荷，需要专用 DRAM 工艺	低，密度高	GB 到百 GB 级	约 10GB/s 级，取决于通道数和代际	约 100ns	通用主存，容量大、成本低，但延迟较高且需要刷新
HBM	本质仍是 DRAM，但多层 die 3D 堆叠，通过 TSV 和 2.5D/先进封装靠近计算芯片	高于 DDR，封装和堆叠成本高	GB 到数十 GB 级	约 100GB/s 到 TB/s 级	约 100ns	面向 GPU/AI 芯片的高带宽近存储，带宽和能效好，但贵且容量相对有限
Flash/SSD	浮栅或电荷俘获晶体管存储电荷，多层 NAND 工艺，非易失	很低，单位容量便宜	TB 级	约 GB/s 级	约 1us 到 100us 级	用于持久化存储，容量大、断电不丢失，但延迟远高于 DRAM
磁盘 (HDD)	磁性盘片和机械磁头，机电系统	最低，单位容量最便宜	TB 到十 TB 级	约 10MB/s 到百 MB/s 级	约 ms 级	超大容量冷/温数据存储，随机访问慢，机械延迟高

拓展：Flip-Flop 容量通常是 KB 级，速度最快，适合寄存器和流水线状态，可并行访问，但每 bit 需要较多晶体管，面积和成本最高。SRAM 容量通常是 MB 级，速度较快、无需刷新，但一个 bit 常用 6 个或更多晶体管，成本仍然较高，并且通常按字或 cache line 访问。DRAM 容量通常是 GB 级，一个 bit 主要由一个晶体管加一个电容保存，密度高、成本低，但访问较慢，读出会破坏电容状态，需要写回和周期性刷新。Flash/SSD 容量通常是 TB 级，非易失、单位容量便宜，但访问延迟远高于 DRAM，适合持久化大容量存储，不适合细粒度高频随机访问。

考点：FF 最快最贵，SRAM 快但容量有限，DRAM/DDR 大且便宜但要刷新，HBM 是更宽、更近、更贵的高带宽 DRAM，Flash/SSD 更大更便宜且非易失但慢得多；“一个 bit 的硬件代价”大致从 FF 到 SRAM 到 DRAM/Flash/HDD 逐级下降，而访问速度和带宽通常也逐级下降。

5.3 SRAM 结构与用途

定义：SRAM 使用交叉耦合反相器保存 1 bit，通常用于寄存器文件、缓存和片上 buffer。

拓展：SRAM 访问快、可随机访问、无需刷新，但单位面积成本高、容量有限。存储阵列由 wordline、bitline、译码器、感放和多路选择器组成，可通过 banking 提供并行访问。

考点：SRAM 适合作片上高速存储；CPU 芯片大面积常被 SRAM cache 占据；banking 可增加并行度但会带来 bank conflict。

5.4 DRAM 组织与访问

定义：DRAM 用电容电荷表示数据，具有高密度、低成本和大容量，但需要刷新，访问延迟较高。

拓展：DRAM 系统可分为 channel、DIMM、rank、chip、bank、row/column。访问一次数据可能经历 ACTIVATE、column access、PRECHARGE 等操作。若访问已打开行是 row buffer hit，延迟较低；若需要切换行则会出现 row conflict。

层次	作用	理解重点
Channel	内存控制器到内存系统的一条独立访问通道	有自己的命令/地址/数据传输路径；多个 channel 可并行访问不同地址
DIMM	插在内存通道上的内存模组	一条内存条可包含多个 rank 和多颗 DRAM chip
Rank	一组对外一起响应的 DRAM chip	多颗 chip 并行凑够 channel 的数据位宽，例如 8 颗 x8 chip 组成 64-bit rank
Chip	单颗 DRAM 颗粒	同一 rank 内的 chip 通常 lockstep 工作，接收同一组命令和地址，各自贡献一部分 bit
Bank	chip 内部相对独立的小存储阵列	不同 bank 可交错访问；每个 bank 有自己的 row/column 结构和 row buffer
Row/Column	bank 内部的二维寻址结构	先打开 row 到 row buffer，再按 column 选择需要的数据

拓展：OS 和程序看到的是连续的虚拟地址或物理地址空间，但真实 DRAM 中会由内存控制器把地址映射到 channel、rank、bank、row、column 等字段；同一次 rank 访问的数据位还会由多颗 chip 并行提供。因此，连续地址在硬件中可能被交错分布到不同 channel、rank、bank 或 chip 中，这种映射通常对 OS 和程序透明。

拓展：channel 是一条相对独立的内存访问通道，通常有自己的命令、地址和数据传输路径；它不是把一个字节按 bit 拆开传输。bus width 表示一个 channel 单次能并行传多少 bit，例如常见 DDR channel 可有 64-bit 数据宽度；multi-channel 则是多条独立通道并行工作。不同 channel 可以同时访问不同地址，但程序员通常不直接指定 channel，而是由内存控制器根据地址映射规则把访问分布到不同 channel；若多个访问落到同一 channel，则需要在该 channel 内排队调度。

拓展：一次 I/O cycle 传输的数据量由 rank 对外数据位宽决定，例如 8 颗 x8 chip 组成的 rank 一次可并行提供 64 bit，也就是 8B。若 cache block 大小为 64B，则传完整个 cache block 需要 8 个 I/O cycle，因为每次传 8B，连续传 8 次才得到 64B。这个过程通常可理解为从已打开的 row buffer 中顺序读取多个 column，而不是“读取 64 bit 需要 8 次”。

拓展：rank、chip、bank 是 DRAM 组织中的不同层次。chip 是单颗 DRAM 颗粒，内部包含多个 bank；bank 是 chip 内部相对独立的小存储阵列，每个 bank 有自己的 row/column 结构和 row buffer。rank 是一组对外一起响应的 DRAM chip，用来凑够 channel 需要的数据位宽，例如 8 颗 x8 DRAM chip 并行工作，就能组成 64-bit 数据宽度的一个 rank。所谓“rank 中并行 chip”，就是同一次内存访问中多颗 chip 同时响应，每颗 chip 贡献一部分 bit，组合成 channel 对外看到的数据。同一个 rank 内的 chip 通常 lockstep 工作，接收同一条命令和同一组 bank/row/column 地址，各自访问自己内部对应的位置；例如 8 颗 x8 chip 会分别返回 8 bit，拼成一次 64-bit 数据访问。可以记为：rank 负责凑总位宽，chip 是实际 DRAM 颗粒，bank 提供 chip 内部的交错访问并行性。

拓展：不同 bank 可交错访问（bank interleaving），意思是内存控制器可以把多个访问分散到不同 bank，让某些 bank 在等待 ACTIVATE、READ/WRITE 或 PRECHARGE 时，其他 bank 继续准备或传输数据。若访问都落在同一个 bank，容易排队并产生 row conflict；若访问分布在不同 bank，就能交错调度，提高总线利用率和有效带宽。不同 bank 还可以各自保持不同的 row buffer 状态，从而增加命中机会。

拓展：row buffer 是 DRAM bank 中“当前被打开的那一整行”的临时缓存。访问某个 row 时，DRAM 先 ACTIVATE 把整行读入 row buffer，再通过 column access 选出需要的数据；若下一次访问仍在同一打开 row 中，就是 row buffer hit，延迟较低；若要访问同一 bank 的另一行，就要先 PRECHARGE 关闭当前行，再 ACTIVATE 新行，形成 row conflict。row 有多长主要取决于 DRAM bank 内部阵列的列数和 row buffer 大小，系统层面还受 rank 中多个 DRAM chip 并行组织影响，通常是几 KB 到十几 KB 量级。

DRAM 访问状态	含义	需要的操作	延迟
Page Hit	目标 row 已经在该 bank 的 row buffer 中打开	直接 column access	最低
Page Closed	该 bank 当前没有打开任何 row	先 ACTIVATE 目标 row，再 column access	中等
Page Miss	该 bank 当前打开的是别的 row	先 PRECHARGE 关闭旧 row，再 ACTIVATE 新 row，再 column access	最高

考点：DRAM bank、row buffer、page hit/miss/conflict 是理解内存性能的关键；channel 是独立内存通路，bus width 是单次传输位宽，二者不要混淆；rank 是一组并行 chip，对外凑够一次访问的数据宽度；64B cache block 在 64-bit 数据总线上需要 8 个 I/O cycle 传完；bank interleaving 通过交错访问不同 bank 提高带宽；Page Hit 最快，Page Closed 需要打开行，Page Miss 需要先关旧行再开新行；DRAM 需要周期性 refresh，刷新带来性能和能耗开销。

5.5 HBM、SSD 与新型存储

定义：HBM 通过堆叠 DRAM die 和宽接口提供高带宽，常与高端 GPU、AI ASIC 和 FPGA 配合使用。

拓展：HBM 每 stack 可提供数百 GB/s 带宽，能耗较低，但成本高、容量相对有限、封装复杂。HBM 与 DDR 都属于 DRAM 技术路线，但定位不同：HBM 通过 3D 堆叠、TSV 和 2.5D/先进封装靠近计算芯片，依靠超宽接口提供高带宽和较低每 bit 传输能耗；DDR 通常作为通用主存或内存条使用，通过主板连线与处理器相连，容量更容易做大、成本更低，但带宽和能效通常不如 HBM。NVMe SSD 和持久内存提供更大容量，但吞吐和延迟远弱于 DRAM/HBM。

考点：HBM = 更贵、更近、更宽、更高带宽；DDR = 更通用、更便宜、更大容量。HBM 解决带宽问题但不直接消除所有内存瓶颈；SSD 适合大容量存储而非细粒度高频访问；AI 训练常受 HBM 容量和带宽共同限制。

5.6 DRAM 刷新与跨层优化

定义：DRAM refresh 是周期性恢复电容电荷的机制，典型要求在固定时间窗口内刷新所有行。

Refresh overhead 影响	说明
性能下降	refresh 期间对应 bank/rank 不能正常服务普通读写请求，请求可能被阻塞或排队
访问延迟增加	若读写请求刚好遇到 refresh，需要等待刷新完成，容易拉高 tail latency
有效带宽降低	refresh 占用内部阵列和命令时隙，减少可用于 read/write 的时间
能耗增加	refresh 需要读出并恢复电容电荷，会产生额外动态能耗
容量越大越严重	DRAM 容量越大，需要刷新的 cell 越多，刷新时间和调度压力越大
温度越高越严重	高温下电容漏电更快，可能需要更频繁 refresh
并行性和 QoS 受影响	all-bank refresh 会暂停多个 bank，per-bank refresh 影响较小但调度更复杂；周期性停顿会影响实时性和服务质量

拓展：不同 DRAM 行的保持时间存在差异，多数行可比最坏情况更少刷新。RAIDR 等方法通过 profiling、binning 和 Bloom filter 减少不必要刷新，体现打破抽象层、利用底层特性优化系统。

考点：DRAM refresh 是必要的可靠性机制，但 refresh overhead 会占时间、占带宽、耗能量，并且随容量增大和温度升高变得更严重；理解底层物理特性可以指导体系结构优化。

5.7 多核协同与内存干扰

定义：多核协同是指多个核心共享缓存、内存控制器、channel、rank、bank 和 DRAM 带宽时，系统需要在吞吐、公平性、响应时间和优先级之间做协调。

拓展：多核程序不只是“多个核各跑各的”，它们会争用共享资源。一个 memory performance hog 可能自己 slowdown 不大，但会占用大量内存带宽、打满 memory controller 队列、造成 bank conflict 和 row buffer 干扰，从而显著拖慢其他程序。High priority app 则是系统更希望保护的关键任务，它可能本身不是最重的内存流量源，但对延迟和稳定性更敏感，因此需要更高的服务质量。

拓展：多核干扰常来自 LLC 争用、内存带宽争用、channel/bank 排队、row buffer conflict、refresh overhead 以及不均匀的请求到达模式。常见缓解方法包括 memory scheduling、bank/channel interleaving、优先级队列、带宽限流、QoS 保护、cache partitioning 和减少不必要的访存。

考点：Memory performance hog 不一定自己最慢，但它能让别人很慢；高优先级应用需要被保护，否则共享内存系统会出现不公平和不可预测的 slowdown。理解多核协同的关键，是把“每个程序的行为”放到“共享内存系统的整体调度”里看。

第六章：GPU 架构与 CUDA 编程模型

6.1 GPU 的定位

定义：GPU 是面向大规模数据并行的高吞吐处理器，通常由 CPU 负责控制和串行部分，GPU 负责大规模并行 kernel。

拓展：CPU 由少量复杂核心和大 cache 优化低延迟，GPU 由大量相对简单核心和高带宽内存优化吞吐。CPU-GPU 协同通常包含主机到设备数据传输、kernel 执行、设备到主机结果传回。

考点：GPU 适合并行度高、计算密集或带宽密集的任务；PCIe/互联传输可能成为 offload 的瓶颈。

6.2 编程模型与执行模型

定义：CUDA/OpenCL 采用 SPMD 编程模型，程序员编写一个 kernel，由大量线程在不同数据上执行相同程序。

拓展：CUDA 中 grid 由多个 thread block 组成，block 内含多个 thread；硬件上 thread block 映射到 SM，thread 进一步组成 warp。编程模型中的 thread 不等于硬件上的独立大核心。

拓展：thread block 是软件/编程模型概念，SM (Streaming Multiprocessor) 是硬件执行单元。一个 thread block 会被整体分配到一个 SM 上，不能跨多个 SM；一个 SM 可以同时驻留多个 thread block，数量受最大线程数、最大 block 数、寄存器文件容量和 shared memory 容量限制。block 内线程可共享同一块 shared memory，并可用 `__syncthreads()` 做 block 内同步；不同 block 通常不能直接同步，也不能直接访问彼此的 shared memory。

拓展：warp 不需要程序员手动划分。程序员只指定 block 中有多少 threads，硬件按 32 个 threads 为一组自动切分成 warp。例如一个 block 有 256 个 threads，就会形成 8 个 warp。这些 warp 都属于该 block 被分配到的同一个 SM，由该 SM 内的 warp scheduler 动态选择就绪 warp 发射执行。

考点：SPMD 是编程模型，SIMT 是硬件执行模型；CUDA 层次结构是 grid -> block -> thread，硬件执行层次是 GPU -> SM -> warp scheduler/执行单元；分配单位是 thread block，不是 warp；warp 是 block 进入 SM 后形成的调度执行单位。

6.3 SIMT 与 Warp

定义：SIMT 是 Single Instruction Multiple Threads，GPU 把多个标量线程组合成 warp，在同一指令流下对不同数据执行。

拓展：warp 通常包含 32 个线程，共享取指和调度控制，从而降低调度开销。硬件底层类似 SIMD 管线，但程序员写的是标量线程代码。warp 可通过交替调度隐藏访存延迟。

拓展：同一个 warp 内的线程共享同一条指令流、同一个调度单位、同一个 active mask 和分支收敛管理；正常无分支分歧时，可近似理解为共享同一个 PC。warp 内 32 个线程执行同一条指令，但使用各自的寄存器、局部变量、谓词条件、访存地址和计算结果。也就是说，warp 共享控制流和调度，不共享每个线程的数据状态。

拓展：SM 以 warp 为单位调度指令，但不是整个 SM 永远只执行同一条指令。一个 SM 内可能有多个 warp scheduler 和多类执行单元，不同 warp 可在不同周期使用 CUDA core、load/store unit、SFU 或 Tensor Core 等资源。若同一 warp 内线程发生分支分歧，硬件会分路径执行，并用 active mask 屏蔽当前路径中不活跃的线程，导致执行效率下降。

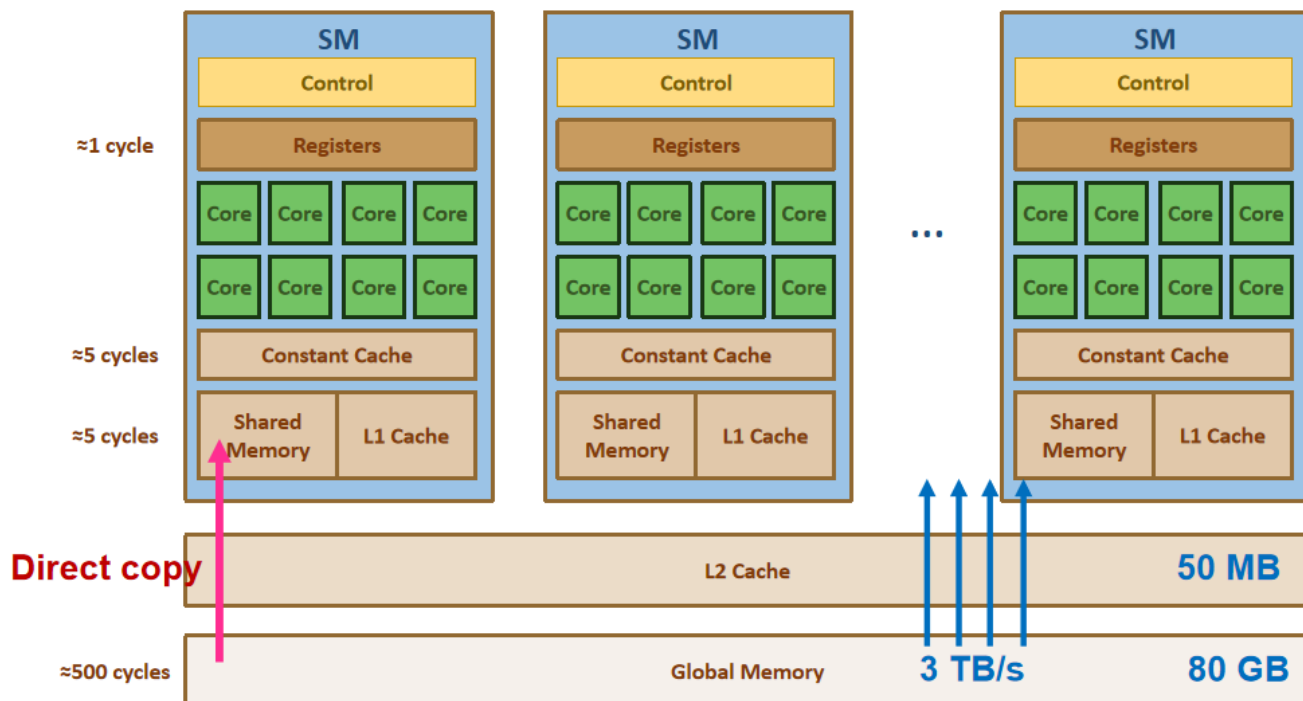
考点：SIMT 不是传统显式 SIMD；warp 是线程调度和执行的基本单位；同 warp 共享指令流，但每个 thread 有自己的寄存器和数据；warp 内线程若执行不同控制路径，会发生分支发散。

6.4 GPU 演进与现代特性

定义：GPU 从早期图形流水线发展为通用并行计算平台，现代 GPU 集成大量 SM、L2 cache、HBM、Tensor Core 和高级数据搬运单元。

拓展：V100、A100、H100 逐代提升核心数、Tensor Core 算力、低精度能力、HBM 带宽和片上共享能力。H100 引入 Tensor Memory Accelerator、分布式 shared memory 等特性以优化数据搬运。

Memory in the GPU Architecture



拓展：一个 SM 可以理解为 GPU 中调度和执行 warps 的基本硬件计算簇，内部包含 warp scheduler、CUDA cores、Tensor Cores、load/store units、SFU、register file、shared memory/L1 cache 等资源。CUDA core 通常指 SM 内执行普通 FP32/INT 标量运算的 ALU lane，它不是一个 CUDA thread，也不是整个 SM；矩阵乘加、访存和特殊函数还会分别使用 Tensor Core、load/store unit 和 SFU 等专用单元。

拓展：同一个 SM 内的 warps/blocks 共享 warp scheduler、CUDA cores、Tensor Cores、load/store units、SFU、L1 cache、instruction cache、register file 总容量、shared memory 总容量和 occupancy slots。需要区分“物理资源共享”和“可见性共享”：register file 物理上属于 SM，但每个 thread 只能访问自己的寄存器；shared memory 物理上在 SM 内，但分配给某个 block 后只对该 block 内线程可见，同一 SM 上不同 block 通常不能直接访问彼此的 shared memory。

考点：GPU 成功不仅来自硬件算力，也来自 CUDA 等稳定编程模型和透明扩展能力；SM 是执行 thread block/warp 的硬件单元，CUDA core 是 SM 内普通标量计算资源，不能把 CUDA core、thread、warp 和 SM 混为一谈。

第七章：GPU 优化

7.1 延迟隐藏与 Occupancy

定义：GPU 通过 warp 级细粒度多线程隐藏全局内存访问等长延迟，occupancy 表示活跃 warp 数占最大可驻留 warp 数的比例。

拓展：当一个 warp 等待内存时，调度器可切换到其他就绪 warp。occupancy 受寄存器使用量、shared memory 使用量、block 大小和硬件上限影响。过低 occupancy 难以隐藏延迟，过高 occupancy 也不一定带来性能提升。

考点：延迟隐藏依赖足够活跃 warp；优化时需要平衡寄存器、shared memory 和并发 block 数。

7.2 全局内存合并访问

定义：Memory coalescing 指同一 warp 中线程访问连续或相邻全局内存地址时，硬件将多个访问合并为少数内存事务。

拓展：合并访问能显著提高带宽利用率；非合并访问会产生更多内存事务，降低有效带宽。数据布局、线程索引和访存 stride 直接影响合并效果。

考点：全局内存优化首先看访问是否 coalesced；相邻线程访问相邻地址通常最友好。

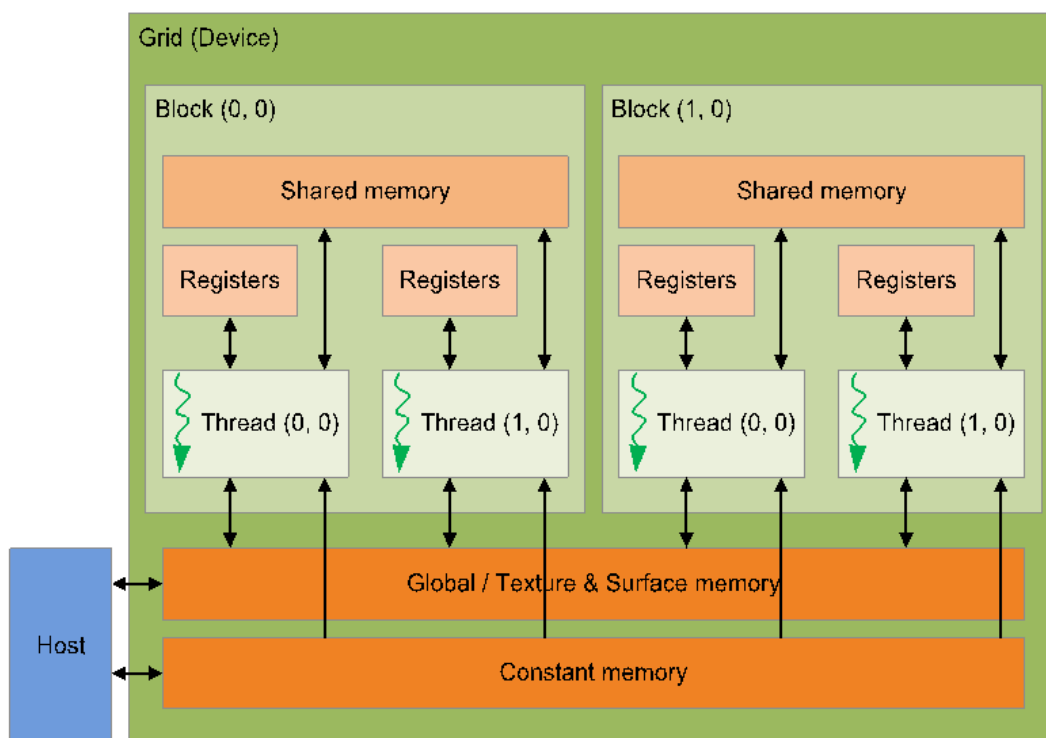
7.3 Shared Memory、Bank Conflict 与 Tiling

定义：Shared memory 是 SM 内由程序员显式管理的片上存储，延迟低、带宽高，但按 bank 组织，可能发生 bank conflict。

拓展：同一 warp 中多个线程访问同一 bank 的不同地址会串行化。Tiling/Blocking 将矩阵或数组分块搬入 shared memory，在片上复用数据，减少全局内存访问。矩阵乘法中常把 A、B 的 tile 加载到 shared memory，再同步后复用计算 C 的 tile。

拓展：矩阵分块的核心思想是让一个 thread block 负责输出矩阵 C 的一个 tile。每轮从 global memory 中加载 A 的一个 tile 和 B 的一个 tile 到 shared memory，使用 `__syncthreads()` 保证 tile 装载完成，然后 block 内线程复用这两个 tile 做多次乘加，累加到各自负责的 C 元素。循环处理完 K 维方向上的所有 tile 后，再把结果写回 global memory。这样可以把大量重复读取 global memory 的操作变成 shared memory 内复用，显著降低带宽压力。

矩阵分块步骤	作用
划分 C 的 tile	每个 thread block 负责一块输出区域
加载 A/B tile 到 shared memory	把慢速 global memory 访问变成片上访问
<code>__syncthreads()</code>	保证 block 内线程都完成装载后再计算
对 tile 做乘加累加	多个线程复用同一批 A/B 数据
写回 C tile	只在最终结果完成后写 global memory



考点：shared memory 优化关注数据复用、bank conflict 和 `__syncthreads()`；tiling 是 GPU 矩阵乘法的核心优化，本质是用片上共享存储换取更少的全局内存访问。

7.4 SIMT 效率、发散与原子操作

定义：SIMT 效率衡量 warp 内活跃线程对执行资源的利用程度；分支发散会让同一 warp 的不同路径串行执行。

拓展：把条件写成 warp 友好的形式可提升利用率，例如让同一 warp 内线程尽量走相同分支。Atomic 操作用于多个线程安全更新同一地址，但冲突会串行化，直方图等场景尤其容易遇到 atomic bottleneck。

拓展：Divergence 发生在同一个 warp 内不同线程走不同控制路径时。硬件会先执行一条路径，同时用 active mask 关闭不属于该路径的线程，再执行另一条路径，最后在汇合点重新收敛。因此 divergence 不会导致结果错误，但会降低并行效率；若一个 warp 中只有少数线程活跃，大量 CUDA cores/lane 会空转。优化时应尽量让同一 warp 内线程走相同分支，或把分支条件设计成按 warp/block 粒度一致。

拓展：Atomic 用于多个线程并发更新同一内存位置时保证读-改-写操作不可被打断，例如 `atomicAdd` 常用于计数器、直方图和稀疏累加。atomic 的问题是冲突越集中，串行化越严重；很多线程更新同一个地址会形成热点。常见优化包括先在 shared memory 中做 block 内局部归约或局部直方图，再少量 atomic 写回 global memory；也可以通过分桶、分片或改变数据布局降低冲突。

考点：减少 warp divergence、减少 atomic conflict、提高活跃线程比例是 GPU 优化重点；divergence 的代价来自同一 warp 多路径串行执行，atomic 的代价来自多个线程争用同一地址。

7.5 CPU-GPU 异步传输

定义：CUDA stream 是按顺序执行的一组命令队列，可用于重叠 CPU-GPU 数据传输和 kernel 执行。

拓展：把输入切分到多个 stream，可让一部分数据传输、另一部分数据计算、第三部分结果回传并行发生。视频处理等独立数据实例适合这种流水化执行。

拓展：CPU-GPU transfer 通常指 host memory 与 device memory 之间的数据传输，例如通过 PCIe 或 NVLink。它的带宽和延迟通常远弱于 GPU 内部 HBM 访问，因此频繁的小数据传输会让 GPU 算力闲置。优化原则是减少传输次数、合并小传输、尽量让数据长期驻留在 GPU 上，并使用 pinned memory、异步 copy 和 stream overlap 把 Host-to-Device、kernel 执行、Device-to-Host 流水化。

传输优化方法	作用
减少 CPU-GPU 往返	避免 kernel 之间频繁把中间结果搬回 CPU
合并小传输	降低启动开销，提高有效带宽
pinned memory	支持更高效的异步 DMA 传输
多 stream 分块流水	让 H2D、计算、D2H 尽可能重叠
数据驻留 GPU	多个 kernel 复用 device memory 中的数据

考点：异步传输提升端到端性能的前提是计算和传输可重叠；CPU-GPU 传输带宽远低于 GPU 内部 HBM 带宽，优化重点是少传、批量传、异步传、重叠传。

第八章：存储层次与 Cache

8.1 存储层次思想

定义：存储层次用多级“越靠近处理器越快越小、越远越慢越大”的结构近似理想存储。

拓展：寄存器、L1、L2、L3、DRAM、SSD 构成典型层次。若程序具有局部性，少量快速存储可缓存近期或邻近数据，让平均访问时间接近快速层。

考点：存储层次的成立依赖局部性；层次越深，平均访问时间越取决于命中率和 miss penalty。

8.2 时间局部性与空间局部性

定义：时间局部性指最近访问过的数据很可能再次访问；空间局部性指最近访问地址附近的数据很可能被访问。

拓展：cache 利用时间局部性保存最近访问 block，利用空间局部性一次搬入连续 cache line。循环、数组遍历和函数局部变量常体现局部性。

考点：cache line 大小主要利用空间局部性；最近使用数据保留在 cache 中主要利用时间局部性。

8.3 AMAT 与层次延迟

定义：平均内存访问时间 AMAT 常写为 $Hit\ Time + Miss\ Rate \times Miss\ Penalty$ 。

拓展：多级 cache 可递归分析：某级 perceived access time 等于本级访问时间加上 miss rate 乘以下一级访问代价。降低 miss rate 不一定总是提升性能，因为更大或更高相联度 cache 可能增加 hit time。

拓展：对第 i 级存储层次，设本级技术固有访问时间为 t_i ，本级命中率为 hr_i ，未命中率为 mr_i ，则感知访问时间可写为

$$T_i = hr_i \cdot t_i + mr_i \cdot (t_i + T_{i+1})$$

又因为 $hr_i + mr_i = 1$ ，所以上式可化简为

$$T_i = t_i + mr_i \cdot T_{i+1}$$

这里的 hr_i 和 mr_i 只针对“在第 $i-1$ 级未命中后落到第 i 级”的那部分访问来定义。这个递推式说明：本级越容易 miss，越依赖下一级的访问代价；若一直递归到最底层，就能得到整条存储层次的总访问延迟。

考点：优化 cache 性能要同时看命中率、命中延迟和 miss penalty；递推式 $T_i = t_i + mr_i T_{i+1}$ 是层次延迟分析的核心；一级 cache 更看重低延迟，低层 cache 更看重容量和带宽。

8.4 Cache 映射方式

定义：Cache block 是 cache 的存储单位，主存地址通常分为 tag、index 和 block offset。

拓展：直接映射中一个主存块只能放到一个 cache 位置，硬件简单但冲突多；全相联中可放任意位置，冲突少但查找昂贵；组相联折中，每个块映射到一个 set，在 set 内可放多个 way。

连接/映射方式	含义	特点
Direct-mapped	一个 chunk 只能放到唯一的一个 cache block 位置	硬件最简单、访问最快，但冲突 miss 最多
Fully-associative	一个 chunk 可以放到 cache 中任意 block 位置	冲突最少，但查找和比较硬件最贵
Set-associative	一个 chunk 只能放到某个 set 中的 N 个 way 之一	现代 cache 最常用的折中方案，兼顾命中率和硬件复杂度

拓展：可以把 direct-mapped 看成“极端简单的一端”，fully-associative 看成“极端灵活的一端”，set-associative 则是实际系统常用的平衡选择。N-way set-associative 中，一个块先映射到唯一的 set，再在该 set 的 N 个 way 中择一放置。

考点：直接映射冲突 miss 多但快；全相联利用率高但硬件复杂；组相联是现代 cache 常用折中。

8.5 Shared Memory 与 Cache 的差异

定义：Cache 是硬件自动管理的数据搬运机制，shared memory / scratchpad / buffer 是程序员或编译器显式管理的片上存储。

拓展：Cache 通常由 SRAM 工艺实现，核心存储单元常是 6T SRAM cell，外加 tag 阵列、比较器、替换逻辑和一致性控制逻辑。它不需要像 DRAM 那样刷新，因而延迟低、访问稳定，也更容易和 CPU/GPU 的逻辑电路集成到同一芯片上。Cache 的高带宽主要来自片上集成、短互连、宽数据通路、banking 和流水化访问；因为离计算单元近，单周期内可并行搬运较多数据，所以吞吐高。

拓展：Cache 的“容量大”是相对寄存器 and 小型 buffer 而言，不是相对 DRAM 而言。SRAM 的单位面积成本高，所以 cache 不可能无限做大；它通常采用多级层次设计，L1 小而快，L2/L3 更大但稍慢，用较小容量尽量命中更多近期/邻近数据。也正因为 cache 是片上 SRAM，它能在高带宽和较低延迟下提供比寄存器大得多的可用容量，但仍明显小于主存 DRAM。

拓展：Cache 降低编程难度，但标签、替换和一致性逻辑复杂；buffer 控制明确、能耗低、性能可预测，但编程复杂。AI 加速器常偏向 Global Buffer/Scratchpad，因为深度学习算子的访存模式相对规则。

考点：CPU 常用 cache 透明管理，AI 加速器常用 buffer 显式管理；这是通用性与能效的典型权衡。

第九章：Cache 策略与一致性

9.1 替换策略

定义：替换策略决定 cache miss 且目标 set 已满时驱逐哪个 block。

拓展：常见策略包括 Random、FIFO、LRU 和伪 LRU。LRU 驱逐最久未访问 block，但高相联度下维护精确 LRU 成本高；某些循环访问模式下 LRU 可能比 Random 更差，出现 set thrashing。

拓展：精确 LRU 的问题在于需要记录同一个 set 内所有 way 的最近使用顺序。若一个 set 有 N 个 way，精确维护通常需要保存完整排序信息，每次命中还要更新整套顺序，硬件比较器和控制逻辑都很复杂，N 越大代价越高。因此实际 cache 往往使用近似 LRU，也叫 pseudo-LRU (PLRU)。近似 LRU 的目标不是精确知道“谁最老”，而是用更低的硬件代价近似找出“很可能最少使用”的 block。

拓展：常见近似 LRU 做法包括树形 PLRU 和 bit-based 近似。树形 PLRU 适用于 2 的幂次 way 数，把 set 内 ways 组织成一棵二叉树，每个内部节点只记录“最近访问的是左子树还是右子树”，替换时沿着“较少被最近访问”的方向往下走，最终找到一个候选 victim。bit-based 近似则为每个 way 维护一个近似年龄位或访问位，通过少量 bit 近似反映最近使用情况。它们都避免了精确 LRU 的全序维护开销。

拓展：近似 LRU 的优点是硬件开销小、时钟路径短、易实现；缺点是 victim 选择不一定真正是最久未用的 way，因此在某些访问模式下命中率略低于理想 LRU。不过对大多数实际工作负载，近似 LRU 已经足够好，是现代高相联度 cache 的常见选择。

考点：Belady OPT 是理论最优，驱逐未来最晚访问的 block，但实际不可实现；现代处理器常用近似 LRU。

9.2 写策略与 miss 分类

定义：写策略包括 write-back/write-through 和 write-allocate/no-write-allocate；miss 可分为 compulsory、capacity、conflict。

拓展：写策略实际上要同时看两个维度。第一个维度是“cache 中命中时，是否立刻写到下一级”；第二个维度是“写 miss 时，是否先把块调入 cache”。因此常见组合是 write-back + write-allocate、write-through + no-write-allocate，也有少量特殊场景使用 write-through + write-allocate。

策略	含义	适用场景
Write-back	写命中时只更新 cache，等脏块被驱逐时再写回下层	写多、局部性好、希望减少带宽压力的通用场景
Write-through	每次写命中都同步更新下层	需要简单一致性、写较少、对下层可见性要求高的场景
Write-allocate	写 miss 时先把块读入 cache，再在 cache 中写	后续可能还会继续写同一块，写具有局部性时更合适
No-write-allocate	写 miss 时不把块调入 cache，直接写下层	一次性写、流式写、大块顺序写时更合适

拓展：常见默认选择是 write-back + write-allocate，因为大多数通用程序对写也有一定时间/空间局部性，先装入 cache 再修改，能减少后续访存并合并写回。write-through + no-write-allocate 更简单，适合控制逻辑简单、对可见性要求高、或写流量不大但希望下层立即可见的场景。若程序是一次性大流量写入、很少再读回，例如某些流式输出或临时缓冲，no-write-allocate 可避免把无复用数据污染 cache。

拓展：从“应用场景”来记更直观。对普通内存写操作，默认常见的是 `write-allocate + write-back`：先把 cache line 装进 cache，再在 cache 里修改，最后等脏块被替换时统一写回，这适合写有局部性的通用程序。对 PCIe/IO、流式输出、一次性大块写入等场景，常用 `write-no-allocate`：写 miss 时不把数据搬进 cache，而是直接写到下层或设备，避免把很快不会再用的数据污染 cache。对需要下层立即可见、写流量较小、或实现逻辑要尽量简单的场景，常用 `write-through`：每次写都同步到 memory。对希望减少内存带宽压力、允许延迟写回的通用场景，常用 `write-back`：先写 cache，等 eviction 再写回 memory。

场景	常见策略组合	原因
普通内存写、写有局部性	write-allocate + write-back	先装入 cache，后续写可复用，减少带宽并合并写回
流式写、一次性大块写、PCIe/IO	write-no-allocate + write-through 或直接写下层	不把无复用数据带进 cache，避免污染和额外搬运
需要下层立即可见、写量不大	write-through	简单、可见性强，便于控制逻辑和调试
希望减少内存带宽、允许延迟写回	write-back	脏块可合并多次写，整体效率更高

考点：write-back 先写 cache，驱逐脏块时写回内存，可合并多次写，通常更高效；write-through 每次写同时更新下层，简单但带宽开销大；write-allocate 适合有写复用，no-write-allocate 适合流式一次性写。

Miss 类型	含义	典型原因	缓解方法
Compulsory miss	第一次访问某个 block 时发生的 miss，也叫 cold miss	数据从未进入过 cache	普通 caching 无法完全避免；可用 prefetching 提前把将要用到的 block 搬入 cache
Conflict miss	cache 总容量还够，但多个 block 被映射到同一个 set/位置而互相挤出	直接映射或低相联度下地址冲突	提高 associativity；使用 victim cache；改进/随机化 indexing；通过软件调整数据布局或访问模式
Capacity miss	工作集大于 cache 可容纳容量，即使全相联也放不下	cache 空间不足、活跃数据集过大	增大 cache；改善替换策略；blocking/tiling 把 working set 切小，使每个计算阶段的数据能放进 cache

拓展：三类 miss 的判断顺序常可这样理解：第一次访问导致的是 compulsory miss；如果不是第一次访问，且全相联同容量 cache 可以避免，则是 conflict miss；如果即使用全相联同容量 cache 也放不下工作集，则是 capacity miss。优化时也要对症下药：compulsory 靠预取，conflict 靠相联度/索引/布局，capacity 靠更大 cache 或把计算分块。

考点：预取可减少 compulsory miss；增大容量减少 capacity miss；提高相联度或改变映射可减少 conflict miss。

9.3 私有 Cache 与共享 Cache

定义：私有 cache 属于单个 core，共享 cache 被多个 core 共同使用。

拓展：私有 cache 延迟低、带宽好，但共享数据可能在多个 cache 中复制，引入一致性问题。共享 cache 具有更高有效容量和动态分配能力，但会出现线程间干扰、带宽竞争和访问延迟增加。

考点：多核系统中 cache 既是性能优化点，也是共享资源竞争和一致性复杂度来源。

9.4 Cache Coherence

定义：Cache coherence 保证多个核心对同一内存地址看到一致的最后写入值。

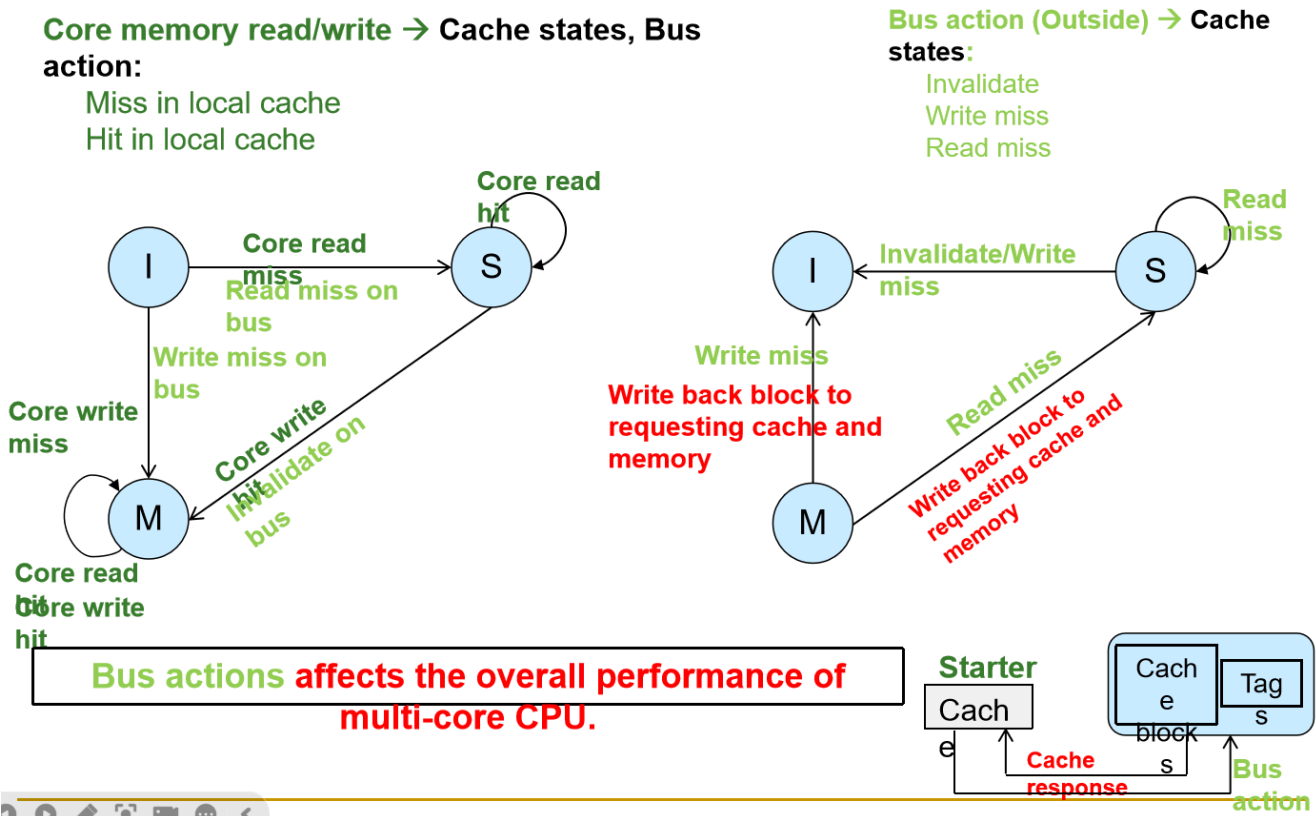
拓展：一致性关注同一地址的操作顺序，不等同于内存一致性模型。硬件通过 cache tag 状态、互联和更新/失效协议维护复制数据。Update 协议广播新值，Invalidate 协议使其他副本失效；现代系统多采用失效思路。

考点：coherence 解决“同一地址多个副本”的正确性；update 适合稳定共享且写少场景，invalidate 更适合写频繁场景。

9.5 MSI、MESI 与目录协议

定义：MSI 协议用 Modified、Shared、Invalid 三种状态维护 cache line；MESI 在此基础上加入 Exclusive 状态，避免独占干净块写入时不必要的总线失效。

State Diagrams for Bus Requests



拓展：MSI 的核心思路是把每个 cache line 的状态限制在三种状态里。M 表示该行被某个 cache 独占且已修改，内存中的副本可能已过期；S 表示该行在多个 cache 中共享且与内存一致；I 表示该行无效。读 miss 时可以把 line 带入 cache 并进入 S，写操作若需要独占权限则会触发失效其他副本，把 line 转成 M。MSI 的正确性来自“同一地址的副本必须同步失效或更新”，它是理解缓存一致性的基础协议。

The Problem with MSI

- A block is not in cache at the beginning. On a read, the block immediately goes to the **"Shared"** state.
- **Problem:** The core issues a bus action invalidate before writing the block to cache, even when only one cache copy exists.

Time	P1 op.	P2 op.	State A in P1	State B in P2	Bus action
t0			I	I	
t1	Read A		S	I	Read miss A
t2	Write A		M	I	Invalidate
t3		Read B	M	S	Read miss B
t4		Write B	M	M	Invalidate

拓展：MSI 的问题在于写一个共享块时，往往需要把其他 cache 中的副本全部失效，然后再写入，这会产生较多总线事务和额外通信。更重要的是，MSI 没有把“独占但干净”的块单独区分出来，所以有时即使块没有被修改，也会在写时引入不必要的失效和获取独占权限的开销。MESI 加入 **E** 状态后，可以把这类块单独标记出来，从而在第一次写时直接转成 **M**，减少一次总线失效。

MESI Protocol

MESI Protocol: Illinois protocol (ISCA, 84)

- ❑ **I**(nvalid): block is not in cache, need to fetch from memory or other cache
- ❑ **S**(hared): in >1 caches, clean, local cores directly reads it w/o bus action
- ❑ **M**(odified): in 1 cache, local core can read/write it w/o bus action
- ❑ **E**(xclusive): in 1 cache, clean, local core reads/writes it w/o bus action

■ **Key Differences** from MSI Protocol:

- ❑ Local core reads block in state **E**, the state holds
- ❑ Local core writes block in state **E** → state **M**, without **bus action**
- ❑ Remote core reads, via **read miss on bus**, block in state **E** → state **S**
- ❑ Remote core writes, via **write miss on bus**, block in state **E** → state **I**

Papamarcos, "A low-overhead coherence solution for multiprocessors with private cache memories," ISCA 1984.

MESI over MSI

	Time	P1 op.	P2 op.	State A in P1	State B in P2	Bus action
MSI:	t0			I	I	
	t1	Read A		S	I	Read miss A
	t2	Write A		M	I	Invalidate
	t3		Read B	M	S	Read miss B
	t4		Write B	M	M	Invalidate
	Time	P1 op.	P2 op.	State A in P1	State B in P2	Bus action
MESI:	t0			I	I	
	t1	Read A		E	I	Read miss A
	t2	Write A		M	I	
	t3		Read B	M	E	Read miss B
	t4		Write B	M	M	

72

拓展：MESI 在 MSI 的基础上加入 **E** (Exclusive) 状态，表示该 cache line 目前只在一个 cache 中存在，但它还是干净的、和内存一致的。这样当这个独占块第一次被写时，不必先广播失效其他副本，就可以直接从 **E** 转到 **M**，减少一次总线事务。**E** 状态最适合“单核先读、随后本核再写”的场景，因此它能有效减少 MSI 在这类场景下的额外开销。上面这张“MESI over MSI”图强调的就是：MESI 相比 MSI 主要多了一个能省掉无谓失效的中间状态。

拓展：local cache 进入 **E** 还是 **S**，不是自己猜的，而是由 coherence 协议在装入时判定。以 snooping MESI 为例，cache 读 miss 后会发出 **BusRd**；如果总线嗅探结果显示没有其他 cache 持有这条 line，那么新装入的 line 可以进入 **E**，表示“独占但干净”；如果有其他 cache 也声明自己持有这条 line，那么新装入的 line 只能进入 **S**，表示“共享”。在 directory 协议里也类似，只是判断依据变成 directory 返回的 sharer 信息：没有 sharer 则可进 **E**，有 sharer 则进 **S**。所以本质上，**E/S** 的区别取决于“这条 cache line 是否还有其他副本”，由协议反馈决定。

协议扩展	新状态	简要含义
Intel i7 MESIF	F = Forward	多个 cache 共享同一干净 line 时，指定其中一个 shared copy 负责响应其他核心的读请求，让数据可从 remote shared cache 转发，而不一定从内存读取
AMD MOESI	O = Owned	某个 cache 持有已修改但可共享的数据，内存副本可能过期；其他核心读时可从这个 owned cache 获得数据，而不必先写回内存

拓展：**F** 和 **O** 都是在 MESI/MESI 类协议上减少不必要内存访问的优化。**F** 解决“多个 shared 副本谁来响应读”的问题，选择一个 forwarder；**O** 允许“脏数据被共享”，让拥有最新脏副本的 cache 直接供数，减少写回内存再读取的开销。

拓展：Snoop 协议通过总线广播请求，总线提供全局顺序但扩展性差。Directory 协议为每个 cache line 维护 sharer 信息，只向相关节点发送消息，适合更多核心和点对点互联。

Directory 角色	作用
Directory entry	为每个 cache line 记录当前 sharer、owner 或状态信息
Sharer list	记录哪些 cache 持有该 line 的共享副本
Owner	若该 line 被修改，则记录当前负责供数或回写的缓存

拓展：Directory-based cache coherence 的核心是“把全局广播改成按需定向发送”。当某个 core 读 miss 时，它先向目录发送请求；目录查询这条 line 的状态，如果没有其他 sharer，就允许该 core 独占并进入 E/M 方向；如果有 sharer，就把请求转发给现有 owner 或共享者，并把新 core 加入 sharer list。写 miss 时，目录会先通知其他 sharer 失效，待权限收拢后再允许写入。这样做的好处是不会让所有核都被广播打扰，特别适合核心数很多、互联是点对点网络、或者广播成本太高的系统。

Snoop 协议点	说明
基本思想	所有 cache 监听总线上的 coherence 请求，谁持有相关 line 就按协议响应
实现方式	通过 bus snooping 观察 BusRd、BusRdX、BusUpgr 等请求并更新本地状态
适用场景	核心数较少、共享总线仍可接受、实现希望简单的系统
优点	设计简单、无需单独目录存储、全局顺序直观
缺点	需要广播给所有 cache，扩展性差，核心数多时总线/互联开销大

拓展：在 snoop 协议里，cache controller 会监听总线消息并根据自己当前的 line 状态作出响应。读请求通常用 BusRd，需要独占写权限时会发 BusRdX 或 BusUpgr；其他 cache 若发现自己也有该 line，就按协议把状态改成 S/I/E/M 中相应的后继状态。Snoop 的好处是逻辑直接、容易理解，但它依赖广播，所以更适合较小规模的多核系统。

考点：directory 协议用“目录条目 + sharer list + owner”代替全局广播，扩展性更好；snoop 简单但靠广播，目录复杂但更适合 many-core。

考点：MESI 比 MSI 减少无谓总线事务；snoop 简单但广播不可扩展；directory 提高扩展性但增加目录存储和协议复杂度。

第十章：Cache Coherence 与 Memory Consistency

10.1 Coherence 与 Consistency 区别

定义：Coherence 规定不同处理器对同一地址操作的顺序；Consistency 规定多处理器系统中不同地址内存操作可以以什么顺序被观察。

拓展：一个系统可以对单个地址保持 coherence，但仍允许不同处理器看到不同地址写入顺序不同。Consistency 是程序员和硬件之间关于内存可见顺序的契约。

考点：coherence 是每个地址的局部一致，consistency 是所有内存操作的全局顺序规则。

10.2 内存屏障

定义：内存屏障用于限制屏障前后 load/store 的重排序，常见类型包括 Load-Load、Load-Store、Store-Store、Store-Load。

屏障类型	限制的重排序	含义
Load-Load	屏障前的 load 不能和屏障后的 load 重排序	保证前面的读先于后面的读被观察或完成
Load-Store	屏障前的 load 不能和屏障后的 store 重排序	保证前面的读不会被后面的写越过
Store-Store	屏障前的 store 不能和屏障后的 store 重排序	保证前面的写先于后面的写对外可见
Store-Load	屏障前的 store 不能和屏障后的 load 重排序	防止后面的读绕过尚未全局可见的写，通常是最强、代价最高的一类

拓展：更强的内存模型要求更多屏障或更少硬件重排序，编程更直观但性能更低；更弱的内存模型允许更多优化，但程序员或编译器需要显式同步。

典型硬件/模型	Load-Load	Load-Store	Store-Store	Store-Load	说明
Sequential Consistency / Dual 386	√	√	√	√	最强，四类屏障都天然满足
TSO / x86-64	√	√	√		允许 store buffer 导致的 Store-Load 重排序，通常需要 mfence 等指令
PSO / ARM（教学模型）	√	√			允许更多 store 相关重排序，通常要靠显式屏障约束
Really weak memory / DEC Alpha					最弱，几乎所有顺序都可能需要显式同步保证

拓展：这张表可以这样理解：模型越强，硬件自动保证的顺序越多，程序越好写，但可优化空间越小；模型越弱，硬件越自由，性能潜力越大，但程序员或编译器必须更频繁地插入屏障来保证正确性。x86 常被看作 TSO，而 ARM、Alpha 一类体系结构通常比 x86 更弱，需要更明确的同步操作。

拓展：经典例子可以帮助判断屏障类型。设初始 `A = B = 0`，Core 1 执行 `A = 1; if (B == 0) print "Hello"`，Core 2 执行 `B = 1; if (A == 0) print "ZJU"`。这个程序的关键是两个 core 都是先 store 再 load，因此在强顺序或 SC 下，不可能出现 Hello 和 ZJU 同时打印；但在 TSO 或更弱模型下，如果 store 先进入 store buffer、尚未对另一方可见，就可能两个 core 都先读到 0，于是两个打印都发生。对于这个例子，真正需要约束的是 Store-Load 重排序，因此只有 Store-Load barrier 或更强的 full fence 才能阻止“双打印”。

屏障类型	对该例是否足够	理由
Load-Load	否	关键顺序不是 load 后接 load
Load-Store	否	关键顺序不是 load 后接 store
Store-Store	否	关键顺序不是 store 后接 store
Store-Load	是	阻止 store 后面的 load 提前或绕过 store 可见性

可能输出	SC / 强模型	TSO / 弱模型（无屏障）
只打印 Hello	可能	可能
只打印 ZJU	可能	可能
都不打印	可能	可能
Hello 和 ZJU 都打印	不可能	可能

考点：屏障控制可见顺序，不只是编译器指令；共享数据保护和锁实现依赖正确的内存顺序；Store-Load 屏障常用于阻止 store buffer 导致的读写乱序，通常代价最大。

10.3 Sequential Consistency

定义：顺序一致性要求多处理器执行结果等价于所有处理器内存操作按某个全局顺序一次执行，且每个处理器自身程序顺序保持不变。

拓展：可把内存想象成一次服务一个 load/store 的开关，所有处理器看到同一服务顺序。顺序一致性易理解、易调试，但限制硬件重排序和 store buffer 优化，性能成本较高。

考点：SC 是强内存模型；它保持单处理器顺序直觉，但可能牺牲大量性能。

10.4 TSO 与 PSO

定义：Total Store Order (TSO) 可理解为 SC 加 store buffer，允许后续 load 绕过尚未全局可见的 store；Partial Store Order (PSO) 进一步允许更多 store-store 重排序或写合并。

拓展：Store buffer 可隐藏写延迟，让核心继续执行，但会让其他核心暂时看不到该写。Write coalescing 合并同一 cache line 的多个写，减少带宽和事务数量，但进一步放松可见顺序。

考点：TSO 比 SC 弱但性能更好；PSO 更弱，需更多同步保证正确性；模型越弱，程序员负担越重。

10.5 同步与互斥正确性

定义：共享数据保护要求多个线程不能无控制地同时更新同一共享状态，通常通过锁、原子指令和内存屏障实现互斥。

拓展：若硬件内存模型和同步原语不可靠，临界区可能失效，程序结果不可重复。正确同步需要 coherence 保证同一地址值一致，也需要 consistency 保证锁和数据访问顺序可见。

考点：cache coherence 不能自动保证并发程序正确；锁、atomic 和 memory fence 是构建正确并行程序的基础。

第十一章：深度学习加速器动机与 DSA 设计

11.1 为什么需要 AI 加速器

定义：AI 加速器是针对深度学习等特定领域工作负载设计的 DSA，目标是在性能、能效和成本上优于通用 CPU/GPU。

拓展：CPU 通用性强但能效较低，GPU 并行吞吐高但仍有通用图形/计算负担，ASIC/AI 处理器牺牲部分通用性换取更高能效和更低延迟。深度学习应用广泛，云端和端侧都需要高效推理/训练硬件。

考点：DSA 的核心是利用应用规律换取效率；AI 加速器主要针对矩阵、向量和张量计算。

11.2 深度学习算子特性

定义：深度学习算子分析主要关注计算特性和访存特性：是否存在固定重复计算模式，以及数据访问局部性和带宽需求。

拓展：卷积层和全连接层以 MAC、矩阵/向量乘法为主；激活函数是向量操作；池化包含局部窗口归约；Transformer 中 Attention 和 FFN 也以矩阵乘法、归约和数据搬运为核心。

考点：FC 和 Conv 等层占据大量计算；深度学习计算模式规则、可并行、可复用，是定制加速器的基础。

11.3 AI 加速器五类设计思想

定义：AI 加速器常见设计思想包括 Global Buffer、专用计算单元、简化控制、低精度支持和算子库/编程栈支撑。

拓展：Global Buffer 减少片外访存；Cube/Matrix Unit、Vector Unit、Scalar Unit 分工执行矩阵、向量和控制任务；控制模块可用多队列管理不同指令流；低精度通过 INT8、FP16、FP8 甚至任意精度提升能效；算子库降低编程难度。

考点：AI 加速器不是简单“更多 ALU”，而是计算、存储、控制和软件栈的整体定制。

11.4 Cache 与 Global Buffer

定义：Cache 是硬件透明管理的自动缓存，Global Buffer 是程序员/编译器显式控制的片上存储。

拓展：深度学习算子访存模式较规则，复杂 cache 的标签、替换和一致性开销未必划算。Global Buffer 单位访问能耗低、性能可控，但需要显式搬运和 tiling，编程难度更高。

考点：CPU 选择 cache 是为通用程序降低编程负担；AI 加速器选择 buffer 是为规则算子提升能效。

11.5 低精度与任意精度

定义：低精度计算用更少 bit 表示权重和激活，以减少存储带宽、计算能耗和片上面积。

拓展：机器学习模型对数值误差有一定容忍度，不同网络、层或任务可能需要不同精度。任意精度方案通过新的数据布局和 bit-serial multiplier 支持灵活 bit 宽，低精度时可接近线性加速并减少内存流量。

考点：低精度有效的根源是模型鲁棒性和带宽/能耗瓶颈；硬件必须提供对应指令和数据通路才能真正受益。

第十二章：DaVinci、TPU 与脉动阵列

12.1 AI 加速器的两大挑战

定义：AI 加速器面临的核心挑战是算力不足和访存代价过大。

拓展：32-bit DRAM 访问能耗远高于 32-bit 加法或乘加，外部内存访问会显著拖慢 AlexNet 等模型。片上 buffer、数据复用和定制矩阵单元是减少外部访存和提升吞吐的基本方法。

考点：AI 加速器优化常同时围绕“减少内存访问”和“提高矩阵/向量并行算力”展开。

12.2 数据流：WS、OS、IS、RS

定义：数据流决定权重、激活和部分和在片上 PE 与 buffer 中如何驻留和复用。

拓展：Weight Stationary 让权重尽量驻留，减少权重读取；Output Stationary 让部分和驻留，减少 psum 读写；Input Stationary 让 activation 驻留；Row Stationary 试图综合复用 filter、activation 和 psum，降低 Global Buffer 访问。

考点：数据流本质是选择复用哪类数据；不同层形状和硬件资源会影响最佳数据流。

12.3 DaVinci AI Core

定义：DaVinci AI Core 是华为昇腾中的 AI 计算核心，典型包含 Cube、Vector、Scalar、MTE/BIU 和片上 Buffer 等模块。

拓展：Cube 模块负责矩阵乘等高吞吐计算；Vector 模块处理激活、池化、类型转换和向量运算；Scalar 模块承担循环、分支和标量控制；MTE/BIU 负责数据搬运和外部总线交互；Event Sync 用于不同指令队列/流水之间同步。

考点：Cube 是算力核心，Vector 是通用向量处理，Scalar 是控制核心，MTE 是数据搬运核心；DaVinci 的优势和难点都来自显式分工与并行。

12.4 TPU 与脉动阵列

定义：脉动阵列用规则 PE 阵列和精心安排的数据流，在 PE 间节奏化传递数据并完成矩阵乘法。

拓展：TPU v1 使用 256x256 MAC 的矩阵乘单元，采用 systolic array 加速推理；TPU v2/v3 支持训练，引入更通用的 vector unit、HBM 和高速互联。脉动阵列可重复利用输入数据，减少访存并提高并发。

考点：脉动阵列优点是规则、并行度高、带宽利用好；缺点是数据编排复杂、对不规则计算适应性较差。

12.5 推理、训练与系统尺度

定义：推理加速器更强调模型驻留、低延迟和高能效，训练系统更强调大规模矩阵算力、HBM 容量/带宽和多芯片互联。

拓展：TPU v2/v3/v4/v5、GB200 NVL72、CloudMatrix 384 等系统显示 AI 计算从单芯片扩展到机柜级系统。训练常受内存带宽和通信限制，推理更容易通过把模型留在片上或近片存储获得数量级能效提升。

考点：训练和推理的瓶颈不同；现代 AI 系统竞争点包括芯片、HBM、互联、编程栈和集群级调度。

第十三章：Cambricon、CANN 与 MindSpore

13.1 Cambricon DLP 架构

定义：Cambricon DLP 面向深度学习处理器，目标是在性能/功耗比和可编程性之间取得平衡。

拓展：DLP-S 单核包含控制模块、计算模块和 SRAM 模块。控制模块包括 IFU、IDU 和指令队列；计算模块支持矩阵、向量和量化；SRAM 通过 DMA 显式管理数据。DLP-M/DLP-C 使用分层多核组织降低 NoC 负载和通信开销。

考点：Cambricon 强调 tensor 粒度指令；DLP-S 是基本计算核，DLP-C/DLP-M 是多核层次结构。

13.2 Cambricon 执行流与 ISA

定义：Cambricon 执行流通常为 IFU 取指、IDU 译码、DMA 搬运 tensor、VFU 预处理、MFU 矩阵计算、VFU 后处理、DMA 写回。

拓展：神经元 tensor 可从 DRAM 进入 NRAM，权重 tensor 可进入 WRAM，矩阵计算结果经 VFU 激活/池化等后处理后写回。ISA 包括控制指令、数据搬运指令、计算指令和逻辑指令。

考点：MLOAD/MSTORE、VLOAD/VSTORE 等负责主存与片上存储交互；矩阵、向量、标量和逻辑指令共同覆盖深度学习算子需求。

13.3 CANN 与算子库

定义：CANN（Compute Architecture for Neural Network）是昇腾面向 AI 芯片的软件栈，向下使能处理器并行加速，向上支持框架和高效算子开发。

拓展：深度学习模型由算子组成，AI 芯片直接编程复杂，因此需要算子库屏蔽硬件细节。昇腾算子库包括 NN、BLAS 等类型；算子有名称、类型、输入输出 tensor、shape、format、weight、bias 等描述。

考点：算子库是连接 AI 框架和 AI 芯片的关键层；算子抽象降低开发复杂度并承载硬件优化。

13.4 TBE、TIK、AI CPU 与 Ascend C

定义：CANN 支持多种算子开发方式：TBE DSL、TIK 和 AI CPU。

拓展：TBE DSL 封装程度高，适合算术逻辑简单或内置支持的算子；TIK 更灵活，适合复杂自定义算子；AI CPU 用 C++ 运行在 AI CPU 上，处理无法高效映射到 AI Core 的场景。Ascend C 采用类 CUDA 的 SPMD 编程范式，用 block_idx 区分多个 AI Core 上的执行实例。

考点：DSL 易用但表达受限，TIK 灵活但开发复杂，AI CPU 通用但性能较低；SPMD 是 NPU/GPU 编程的重要共同范式。

13.5 GE 图引擎与图优化

定义：GE 图引擎负责计算图准备、拆分、优化、编译、加载和执行，是 CANN 中连接框架图和运行时任务的核心组件。

拓展：图准备包括 shape 推导和并行拆分；图优化包括 CSE、权值格式转换、图聚合和算子融合；图编译生成资源分配和 task。UB 融合把多个小算子在片上 buffer 级别融合，减少主存读写和上下文切换。FlashAttention 类优化说明理论计算复杂度不变时，也可通过降低内存复杂度显著提速。

考点：算子融合的核心收益是减少中间结果落内存；图优化把框架层表达转化为更适合硬件执行的任务。

FlashAttention 可作为“减少访存复杂度也能提速”的例子理解，但第 15 章 FlashAttention 具体算法细节不作为考试复习重点。

13.6 MindSpore 关键技术

定义：MindSpore 是面向 AI 应用的框架，围绕自动并行、二阶优化、动静态图统一和 AI+科学计算等方向进行系统优化。

拓展：自动并行面向超大模型和超大数据集，将数据并行、模型并行和算子级切分结合；二阶优化可减少迭代次数但计算复杂；动静态图结合兼顾调试灵活性和部署性能；AI+科学计算利用自动微分、异构硬件和稀疏张量代数加速电磁、气象、分子动力学等应用。

考点：框架的价值不只是封装算子，还包括并行策略、编译优化、自动微分和领域应用支撑。

13.7 Wafer Scale Engine 与片上 Scratchpad

定义：Cerebras Wafer Scale Engine 是晶圆级 AI 加速器，通过大量核心、片上 SRAM 和二维 mesh 构建超大规模片上计算系统。

拓展：WSE 使用分布式 scratchpad SRAM，每个 tile 有本地片上存储并通过 mesh 互联。相比传统多芯片系统，晶圆级设计试图减少跨芯片通信和外部访存开销，但制造、良率、散热和编程系统都更复杂。

考点：WSE 体现 AI 芯片向更大片上存储、更紧密互联和系统级封装演进；scratchpad 与显式数据管理仍是核心思想。

第十四章：Parallel Training 与通信模式

14.1 分布式训练为什么需要并行

定义：分布式训练用多块 GPU/NPU 协同训练模型，解决单卡算力、显存容量、数据规模和训练时间不足的问题。

拓展：训练一层线性层可分为 forward、backward 和 weight update。Forward 计算 $y = xw$ ，backward 需要计算权重梯度 dw 和激活梯度 dx ，优化器再根据梯度更新权重。大模型训练的显存主要被参数、梯度、优化器状态和 activation 占用；混合精度训练中还常保留 FP32 master weights、梯度和优化器状态，因此参数量放大后单卡很快放不下。

考点：并行训练不是只有“把数据分给多卡”；要分别看算力扩展、显存扩展和通信开销。模型变大时需要 model/tensor/pipeline parallel，数据变大或吞吐不足时常用 data parallel。

14.2 Data Parallel 与 Ring AllReduce

定义：Data parallel 中每个 worker 保存完整模型，处理不同 mini-batch 切片；反向传播后各 worker 需要同步梯度，保证权重更新一致。

拓展：每张卡独立 forward/backward 得到本地梯度，然后对梯度做 AllReduce，得到所有 worker 梯度的和或平均值，再本地执行相同 weight update。Data parallel 的核心通信是梯度 AllReduce，常可和反向传播中其他层的计算重叠。

Ring AllReduce 项	结论
数据大小	总数据量 M ，切成 N 份，每份 M/N
GPU 数	N
阶段 1	ReduceScatter, $N-1$ 轮，每轮每卡发送并接收 M/N
阶段 2	AllGather, $N-1$ 轮，每轮每卡发送并接收 M/N
总轮数	$2(N-1)$ 轮
每卡通信量	发送约 $2(N-1)M/N$ ，接收约 $2(N-1)M/N$
全系统链路传输量	约 $2(N-1)M$ 量级，具体看按发送量还是双向收发量统计

考点：Ring AllReduce 分成 ReduceScatter 和 AllGather，不是一轮完成。考试若问轮数，答 $2(N-1)$ ；若问每卡通信量，答每阶段 $(N-1)M/N$ ，两阶段合计 $2(N-1)M/N$ 。

14.3 Model Parallel 与 Pipeline Parallel

定义：Model parallel 把模型参数或层分给多个 worker。Inter-layer model parallel 常称 pipeline parallel，每个 worker 负责一段连续层；intra-layer model parallel 常称 tensor parallel，每个 worker 负责同一层中的一部分矩阵。

拓展：Pipeline parallel 的 forward 传 activation，backward 传 activation gradient。若只有一个 mini-batch，流水线首尾会有大量 bubble；GPipe 用 K 个 micro-batch/sub-minibatch 填充流水线。若有 N 个 pipeline stage 和 K 个 micro-batch，课件给出的 forward/backward step 数为 $2(N+K-1)$ ，总 step-slots 为 $2N(N+K-1)$ ，idle step-slots 为 $2N(N-1)$ ，空闲比例为 $(N-1)/(N+K-1)$ 。

考点：Pipeline parallel 的通信是相邻 stage 间点对点传 activation / activation gradient，通常难以完全隐藏；主要问题是 bubble、负载不均衡和层间依赖。

14.4 Tensor Parallel：按行/按列切权重

定义：Tensor parallel 把同一层权重矩阵切到多个 worker 上，让多个 worker 共同完成一层计算。设线性层为 $Y = XW$ 。

切分方式	权重怎么切	Forward 输出形态	Forward 后常见通信	Backward 对应关系
Row-wise partitioning	按课件口径，每个 worker 持有一部分 weight rows	每个 worker 计算一部分 output activations	Forward 层间需要 AllGather，把分片 activation 拼成下一层所需输入	backward 中变成 col-wise 通信形态，常见为 ReduceScatter
Column-wise partitioning	按课件口径，与 row-wise 形成互补切法	多个 worker 对下一层所需输入产生需要归约/分散的结果	Forward 层间需要 ReduceScatter	backward 中变成 row-wise 通信形态，常见为 AllGather

按课件直接记通信：Row-wise forward 是 AllGather，backward 是 ReduceScatter；Column-wise forward 是 ReduceScatter，backward 是 AllGather。若题目要求解释原因，先判断该层输出是“activation 分片需要拼接”还是“部分结果需要归约并分散”。

考点：考试按 PPT 术语作答时，优先使用 slide 74 的总结：Row-wise: Fwd AllGather, Bwd ReduceScatter; Col-wise: Fwd ReduceScatter, Bwd AllGather。不要只背 collective 名字，还要说明通信对象是 activation 或 activation gradient。

14.5 Alternating Partitioning

定义：Alternating partitioning 指相邻层交替采用 column-wise 和 row-wise 切分，使上一层产生的分片输出可以直接作为下一层的分片输入，从而减少层间同步。

拓展：Transformer block 中常见策略是成对使用 column-wise + row-wise，Attention 和 MLP 都可以用类似思路。根据课件，row-wise 与 column-wise 交替时，相邻两层之间可以不通信，因为 worker i 产生的分片输出正好作为下一层 worker i 的输入；但每两层仍需要一次 AllReduce，在 forward 和 backward 中都是如此。这样相比每层都同步，可以大约减少一半同步次数。

考点：Alternating partitioning 的目的不是消灭所有通信，而是减少不必要的层间 activation 同步；考试要能说明“上一层输出分片正好匹配下一层输入分片”这一点。

14.6 通信模式总表

并行方式	计算/存储切分	主要通信	是否容易隐藏	主要难点
Data parallel	每卡完整模型, 不同数据	梯度 AllReduce	相对容易和 backward 重叠	大 batch 收敛、梯度同步带宽
Pipeline parallel	不同 worker 负责不同层段	activation 与 activation gradient 点对点传递	较难	bubble、负载均衡、层间依赖
Tensor parallel	同一层矩阵按行/列切	AllGather、ReduceScatter、AllReduce	较难	collective 频繁、切分方式影响后续层
ZeRO	优化器状态、梯度、参数分片	更多 collective	取决于实现	用通信换显存, 训练更大模型

考点：通信题要写清楚“传什么”：data parallel 传梯度；pipeline parallel 传 activation / activation gradient；tensor parallel 传 activation 分片、部分和或梯度分片。

第十五章：FlashAttention 与前沿优化

15.1 复习优先级说明

定义：第 15 章中的 FlashAttention 部分不考，明显偏前沿研究介绍的内容也不作为主要复习对象。

拓展：可以保留一个宏观理解：某些 AI 系统优化不改变理论 FLOPs，却能通过减少 HBM 读写、改善 tiling 和片上复用显著提速。这一思想与前面 Global Buffer、Roofline、memory-bound 优化是一致的。

考点：不需要背 FlashAttention 的分块 softmax 细节、算法推导和具体实现流程。若考试涉及相关思想，通常只需答“降低访存、提高片上复用、改善 memory-bound kernel”。

第十六章：总复习与四类例题模板

16.1 Roofline 例题模板

定义：Roofline 用 $Performance \leq \min(Peak\ compute, AI * Peak\ bandwidth)$ 判断算子受计算峰值还是内存带宽限制，其中 $AI = ops / bytes$ 。

做题步骤：

- 1. 计算每个算子的算术强度 $AI = OPS / memory\ bytes$ 。
- 2. 在图上用点 (AI, measured performance) 表示实际性能。
- 3. 用题目给出的点或 roofline 转折点反推峰值算力和内存带宽：水平 roof 是 Peak compute，斜线斜率是 Peak bandwidth，交点 $AI^* = Peak\ compute / Peak\ bandwidth$ 。
- 4. 比较实测性能与理论上界：接近 roofline 说明优化较充分；远低于 roofline 说明还有计算利用率、访存模式、并行度或同步开销问题。

考点：AI 大不一定实测性能高；真正判断 compute-bound / memory-bound 要看 AI 相对 ridge point 的位置。优化建议也要对应瓶颈：memory-bound 优先提高 locality、减少 bytes、用 tiling/cache/shared memory/global buffer；compute-bound 优先提高并行度、向量化、Tensor Core/Cube 利用率。

16.2 Pipelined CPU / Tomasulo 例题模板

定义：此类题要求根据指令依赖、功能单元延迟、是否乱序发射、是否 ROB 精确提交，填每条指令在每个周期的 F/D/E/R/W 状态。

做题步骤：

1. 先列依赖：RAW 是真实依赖，必须等生产者结果；WAR/WAW 是名字相关，可由 register renaming 消除。
2. 画 dataflow graph：节点是指令，边是值依赖。只画真实数据流，不把单纯同名寄存器误画成 RAW。
3. 按硬件约束排周期：功能单元数量、延迟、是否 fully pipelined、issue/dispatch 宽度、是否按序 commit。
4. 若有 ROB：执行可乱序完成，但 w/commit 必须按程序顺序；若队头未完成，后面完成的指令只能在 ROB 等待。
5. 若是 in-order dispatch 且无可绕过机制：前面等待源操作数的指令会阻塞后续独立指令进入执行。

考点：fully pipelined 表示同一个功能单元可每周期接收新操作，但每条操作仍有固定 latency。精确异常要求按序退休，不等于按序执行。

16.3 Performance Analysis 例题模板

定义：程序执行时间常用 $\text{CPU time} = \text{Instruction count} * \text{Average CPI} * \text{Clock cycle time}$ 。多周期处理器的平均 CPI 是各类指令 CPI 按比例加权。

做题步骤：

1. 理想 CPI： $\text{Average CPI} = \sum(\text{指令比例}_i * \text{CPI}_i)$ 。
2. AMAT： $\text{Average memory access time} = \text{hit time} + \text{miss rate} * \text{miss penalty}$ 。注意 miss penalty 要和 clock cycle time 统一单位。
3. 若题目给每条指令平均访存次数：额外内存停顿通常为 $\text{Instruction count} * \text{memory accesses per instruction} * \text{miss rate} * \text{miss penalty cycles}$ 。
4. 总时间：把基础执行周期和 miss penalty 周期相加，再乘 clock cycle time；或先换成 ns 后相加，但全题单位必须一致。

考点：hit time 若已经包含在基础 load/store CPI 中，不要重复加；若题目明确“理想条件 hit rate 100%”的 CPI 不包含 miss penalty，则 miss 只加额外 penalty。看到 100ns 和 2ns clock，应先换算成 50 cycles。

16.4 Cache 例题模板

定义：Cache 题主要考地址如何映射到 block、set 和 tag，以及 hit/miss 类型判断。

做题步骤：

1. 由容量 C 和块大小 b 算 cache block 数： $B = C / b$ 。
2. 由相联度 N 算 set 数： $S = B / N$ 。Direct mapped 是 $N=1$ ，fully associative 是 $S=1$ 。
3. 对每个地址算 memory block number： $\text{block_addr} = \text{floor}(\text{address} / b)$ ；byte offset 是 $\text{address} \bmod b$ 。
4. 若是 set-associative/direct mapped： $\text{set_index} = \text{block_addr} \bmod S$ ， $\text{tag} = \text{floor}(\text{block_addr} / S)$ 。
5. 按访问序列逐步更新 cache 状态；set 满时按题目给定替换策略，如 LRU，选择 victim。
6. miss 类型：第一次访问某个 memory block 是 compulsory miss；若同容量 fully associative 可命中而当前组织 miss，是 conflict miss；若 fully associative 同容量也放不下工作集导致 miss，是 capacity miss。

考点：地址是 byte-addressable 时，块大小 4B 意味着 0x8 和 0xB 在同一 block。分类 miss 时不要只看 direct mapped 的现象；capacity/conflict 的判断要和 fully associative 同容量对照。

16.5 最后一轮复习顺序

建议顺序：先看第 16 章总复习，做完 roofline、pipelined CPU、performance analysis、cache 四类例题；再回看第 3 章 Tomasulo、第 8-10 章 cache/coherence/consistency、第 14 章 parallel training；最后快速扫第 11-13 章 AI 加速器、DaVinci/TPU/CANN/MindSpore 的概念题。

考点：考试更可能考“能否根据新数据推导/判断”，而不是复述某一页 PPT。复习时每个公式都要能代入数字，每个表结构都要能按周期更新。